



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ _____

КАФЕДРА _____ СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ _____

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
По дисциплине «ММО в АСОИУ»
«Реализация алгоритма Policy Iteration»

Студент ИУ5-22М
(Группа)

(Подпись, дата)

С.С.Костин
(И.О.Фамилия)

Преподаватель

(Подпись, дата)

Ю.Е.Гапанюк
(И.О.Фамилия)

СОДЕРАЖНИЕ

СОДЕРАЖНИЕ	2
1. ЦЕЛЬ И ЗАДАНИЕ	3
2. ХОД РАБОТЫ	4
2.1. Выбор среды	4
2.2. Реализация оценки политики	4
2.3. Реализация итерации политики	5
2.4. Визуализация оптимальной функции ценности.	6
ЗАКЛЮЧЕНИЕ	8
ПРИЛОЖЕНИЕ А ЛИСТИНГ ПРОГРАММЫ	10

1. ЦЕЛЬ И ЗАДАНИЕ

Цель лабораторной работы: ознакомление с базовыми методами обучения с подкреплением.

Задание. На основе рассмотренного на лекции примера реализуйте алгоритм Policy Iteration для любой среды обучения с подкреплением (кроме рассмотренной на лекции среды Toy Text / Frozen Lake) из библиотеки Gym (или аналогичной библиотеки).

2. ХОД РАБОТЫ

2.1. Выбор среды

Для демонстрации алгоритма итерации политики выбрана среда CliffWalking-v1 из библиотеки Gymnasium. Это классическая задача с дискретным пространством состояний и действий, хорошо подходящая для иллюстрации методов динамического программирования. Среда представляет собой сетку размером 4×12 (4 строки, 12 столбцов). Агент начинает из левого нижнего угла («S») и должен достичь правого нижнего угла («G»). Нижняя строка между стартом и целью – это «обрыв» (cliff). Наступление на обрыв даёт штраф -100 и мгновенно возвращает агента на старт. Каждый обычный шаг стоит -1. Цель терминальна – после её достижения эпизод заканчивается.

Пространство состояний и действий.

Всего в среде 48 состояний (4×12). Доступны 4 действия: вверх (0), вправо (1), вниз (2), влево (3). Переходы детерминированы за исключением границ сетки – попытка выйти за пределы поля оставляет агента на месте. Динамика среды полностью известна, что позволяет использовать метод итерации политики, опирающийся на модель переходов.

2.2. Реализация оценки политики

Функция `policy_evaluation` итеративно решает систему уравнений Беллмана для заданной политики. Для каждого состояния перебираются все возможные действия в соответствии с вероятностями текущей политики, а для каждого действия – все возможные следующие состояния (в данной среде переход детерминирован, поэтому вероятности равны 1). Функция ценности обновляется по формуле:

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{\{s', r\}} p(s', r|s, a) [r + \gamma \cdot V(s')]$$

Итерации продолжаются до тех пор, пока максимальное изменение $V(s)$ на итерации не станет меньше заданного порога θ . Коэффициент дисконтирования γ взят равным 0.99.

Реализация улучшения политики. Функция `policy_improvement` для каждого состояния вычисляет ценность каждого действия как:

$$q(s, a) = \sum_{\{s', r\}} p(s', r | s, a) [r + \gamma \cdot V(s')]$$

Затем политика обновляется: все оптимальные действия (с максимальным q) получают равные вероятности, остальные – нулевые. Если для всех состояний argmax новой политики совпадает с argmax старой, политика считается стабильной (найдена оптимальная).

2.3. Реализация итерации политики

Основной алгоритм циклически выполняет два шага: сначала оценивает текущую политику (вычисляет V), затем улучшает её на основе полученной V . Процесс повторяется до тех пор, пока политика не перестанет меняться. В реализации использован порог сходимости $\theta = 10e-3$ для оценки политики (увеличен по сравнению со стандартным $10e-8$ для ускорения, так как на практике точная сходимость V не требуется для последующего улучшения). Алгоритм завершился за 5 итераций, что типично для данной среды (рис. 1).

```
print("Оптимальная функция ценности (первые 5 состояний):\n", optimal_v[:5])
print("Оптимальная функция ценности (последние 5 состояний):\n", optimal_v[-5:])

Итерация 1: Политика стабильна = False
Итерация 2: Политика стабильна = False
Итерация 3: Политика стабильна = False
Итерация 4: Политика стабильна = False
Итерация 5: Политика стабильна = True
Итерация политики завершена за 5 итераций.

Оптимальная политика (первые 5 состояний):
[[0.5 0. 0. 0.5]
 [0. 0. 0. 1. ]
 [0. 0. 0. 1. ]
 [0. 0. 0. 1. ]
 [0. 0. 0. 1. ]]
Оптимальная функция ценности (первые 5 состояний):
[-99.90167837 -99.90266158 -99.90363497 -99.90459862 -99.90555263]
Оптимальная функция ценности (последние 5 состояний):
[-99.91107968 -99.91196888 -99.91284919 -99.9137207 -99.91458349]
```

Рисунок 1 – Результаты итераций политики

Визуализация оптимальной политики. Оптимальная политика отображена в виде сетки 4x12:

- Целевое состояние (G) – правый нижний угол.
- Состояния обрыва (C) – вся нижняя строка между стартом и целью.
- Остальные состояния показывают оптимальное действие: стрелки вверх ($\uparrow$), вправо ($\rightarrow$), вниз ($\downarrow$), влево ($\leftarrow$).

Полученная политика ожидаема и рациональна (рис. 2): агент в верхних строках движется вправо ($\rightarrow$), чтобы быстрее достичь правой стороны, затем спускается вниз ($\downarrow$) и идёт влево ($\leftarrow$) к цели, держась подальше от обрыва. В нижней строке непосредственно перед обрывом агент всегда выбирает движение вверх ($\uparrow$), чтобы избежать падения. В самом левом столбце агент идёт вправо ($\rightarrow$). На стартовой позиции агент движется вправо ($\rightarrow$).

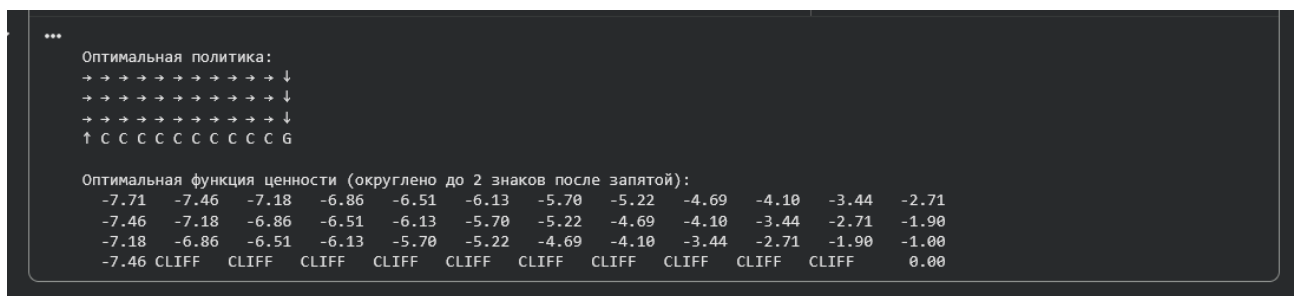


Рисунок 2 – Полученная политика

2.4. Визуализация оптимальной функции ценности.

Функция ценности $V(s)$ отображает ожидаемое суммарное дисконтированное вознаграждение от каждого состояния при следовании оптимальной политике. Значения округлены до двух знаков:

- Цель (G) имеет ценность 0 – логично, так как процесс завершается.
- Состояния обрыва (C) отмечены как «CLIFF» – они не имеют осмысленной ценности, поскольку попадание туда мгновенно возвращает агента на старт с большим штрафом.

- Наиболее высокие (наименее отрицательные) ценности – вблизи цели (около $-1 \dots -3$). Вдали от цели ценности сильно отрицательные (до -8), отражая накопленный штраф за длинный путь.
- Хорошо виден градиент: ценность возрастает (становится менее отрицательной) по мере приближения к цели. В верхней строке ценности примерно на 0.5 единиц ниже (хуже), чем в соответствующей колонке нижней строки, так как путь длиннее.

Интерпретация результатов. Найденная политика – классический оптимальный маршрут для CliffWalking: агент идёт вверх от старта, затем вправо по верхней границе, спускается в крайнем правом столбце и движется влево к цели, полностью избегая обрыва. Альтернативный маршрут (вдоль самого края обрыва) невыгоден, так как риск падения перевешивает экономию нескольких шагов (каждое падение даёт -100 против -1 за шаг). Функция ценности количественно подтверждает эту логику: состояния, расположенные на безопасном пути, имеют заметно более высокие (меньшие по модулю) значения, чем состояния, удалённые от него.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы был реализован и протестирован алгоритм итерации политики (Policy Iteration) – один из базовых методов динамического программирования в обучении с подкреплением. В качестве среды выбрана задача CliffWalking-v1 из библиотеки Gymnasium.

Основные результаты:

- Алгоритм успешно нашёл оптимальную политику за 5 итераций. Это демонстрирует эффективность метода на задачах с известной моделью переходов и небольшим дискретным пространством состояний.
- Оптимальная политика визуализирована в виде стрелок. Агент движется по безопасному маршруту, полностью избегая области обрыва: старт → вверх → вправо по верхней границе → вниз в крайнем правом столбце → влево к цели.
- Оптимальная функция ценности также визуализирована. Её значения убывают (становятся более отрицательными) по мере удаления от цели, что отражает накопленный штраф за каждый шаг (-1). Наиболее высокие ценности – вблизи цели, наиболее низкие – вдали от неё. Ценность цели равна 0.

Метод итерации политики гарантированно сходится к оптимальному решению за конечное число итераций. В данном случае сходимость наступила быстро благодаря простой логике задачи. Основное ограничение метода – необходимость знания полной модели среды (вероятностей переходов), что в реальных задачах часто недоступно. Однако для учебных целей и задач с известной динамикой метод остаётся важным эталонным алгоритмом.

Практическая значимость работы заключается в понимании принципов работы итерации политики: как чередование оценки текущей стратегии и её улучшения позволяет последовательно приближаться к оптимуму. Полученные навыки могут быть перенесены на более сложные алгоритмы, такие как итерация

ценности или методы обучения с подкреплением без модели (Q-learning, SARSA).

Реализованный алгоритм корректно решает задачу поиска оптимального поведения в среде CliffWalking, что подтверждается визуализацией политики и функции ценности. Метод итерации политики продемонстрировал быструю сходимость и высокую точность на данной задаче.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ ПРОГРАММЫ

```
# -*- coding: utf-8 -*-
"""Костин ММО ЛР5 ИУ5-22М.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/1L9F9vq9J1N8uwg7t2YQW5gagJJw0rshe

### Итерация Политики для среды CliffWalking-v0

Этот ноутбук демонстрирует алгоритм Итерации Политики, фундаментальный метод
в обучении с подкреплением для нахождения оптимальной политики для заданного
Марковского процесса принятия решений (МППР). Мы применим его к среде `CliffWalking-
v0` из библиотеки `gymnasium`.

**Описание среды (`CliffWalking-v0`):**
Агент начинает с обозначенного состояния 'S' (старт) и стремится достичь
состояния 'G' (цель). В среде есть область 'скалы'. Если агент наступает на скалу,
он немедленно возвращается в начальное состояние. Перемещение в нескалистое
состояние приносит вознаграждение -1. Падение со скалы приносит вознаграждение -100
и сбрасывает в начальное состояние.

### Шаги:
1. **Установить `gymnasium`:** Библиотека для сред.
2. **Инициализировать среду:** Загрузить `CliffWalking-v0`.
3. **Реализовать Оценку Политики:** Вычислить функцию ценности для данной
политики.
4. **Реализовать Улучшение Политики:** Обновить политику на основе функции
ценности.
5. **Реализовать Итерацию Политики:** Итерировать между оценкой и улучшением
до сходимости политики.
6. **Визуализировать результаты:** Отобразить оптимальную политику и функцию
ценности.
"""

# Install gymnasium for the environment
!pip install gymnasium

import numpy as np
import gymnasium as gym

# Создание среды CliffWalking-v1 (обновлено с v0)
env = gym.make('CliffWalking-v1')

# Получение количества состояний и действий
n_states = env.observation_space.n
n_actions = env.action_space.n

print(f"Количество состояний: {n_states}")
print(f"Количество действий: {n_actions}")

# Инициализация случайной политики (равновероятная для всех действий)
# policy[s, a] дает вероятность выполнения действия 'a' в состоянии 's'
policy = np.ones((n_states, n_actions)) / n_actions
```

```
# Инициализация функции ценности (V) нулями
V = np.zeros(n_states)
```

```
print("Начальная политика (первые 5 состояний):\n", policy[:5])
print("Начальная функция ценности (первые 5 состояний):\n", V[:5])
```

```
"""### 3. Реализация Оценки Политики (Policy Evaluation)
```

На этом шаге мы оцениваем, насколько хороша текущая политика, вычисляя функцию ценности состояния $V(s)$ для каждого состояния. Функция ценности $V(s)$ представляет собой ожидаемую сумму дисконтированных вознаграждений, начиная со состояния s и следуя данной политике.

```
"""
def policy_evaluation(policy, env, gamma=1.0, theta=1e-8):
    """
    Выполняет оценку политики.
    """
    V = np.zeros(env.observation_space.n)
    while True:
        delta = 0
        for s in range(env.observation_space.n):
            # Если это целевое состояние, его ценность всегда 0
            if s == env.observation_space.n - 1:
                continue

            v = 0
            for a, action_prob in enumerate(policy[s]):
                for prob, next_state, reward, terminated in
env.unwrapped.P[s][a]:
                    # Уравнение Беллмана:  $V(s) = \sum(\pi * p * (r + \gamma * V(s')))$ 
                    # В gymnasium CliffWalking terminated=True только на Goal
                    v += action_prob * prob * (reward + gamma * V[next_state])

            delta = max(delta, np.abs(v - V[s]))
            V[s] = v
            if delta < theta:
                break
    return V
```

```
"""### 4. Реализация Улучшения Политики (Policy Improvement)
```

На этом шаге мы улучшаем текущую политику, делая ее жадной по отношению к функции ценности, вычисленной на предыдущем шаге. Для каждого состояния мы выбираем действие, которое максимизирует ожидаемое вознаграждение, плюс дисконтированное значение следующего состояния. Если несколько действий дают одинаковое максимальное значение, мы можем распределить вероятность между ними.

```
"""
def policy_improvement(current_policy, env, V, gamma=1.0):
    """
    Улучшает политику на основе функции ценности.
    """
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    new_policy = np.zeros((n_states, n_actions))
    policy_changed = False

    for s in range(n_states):
        if s == n_states - 1: # Пропускаем целевое состояние
            new_policy[s] = current_policy[s]
```

```

        continue

    old_actions = np.copy(current_policy[s])
    action_values = np.zeros(n_actions)

    for a in range(n_actions):
        for prob, next_state, reward, terminated in
env.unwrapped.P[s][a]:
            action_values[a] += prob * (reward + gamma * V[next_state])

    # Находим лучшие действия (аргмакс с учетом точности float)
    best_value = np.max(action_values)
    best_actions = np.where(np.isclose(action_values, best_value,
atol=1e-8))[0]

    new_policy[s, best_actions] = 1.0 / len(best_actions)

    if not np.allclose(old_actions, new_policy[s]):
        policy_changed = True

    return new_policy, not policy_changed

```

"""### 5. Реализация Итерации Политики (Policy Iteration)

Итерация Политики объединяет два предыдущих шага: Оценку Политики и Улучшение Политики. Алгоритм итеративно выполняет эти шаги, пока политика не станет стабильной, то есть пока дальнейшее улучшение политики не приведет к ее изменению. В этот момент мы найдем оптимальную политику и соответствующую ей оптимальную функцию ценности.

"""

```

def policy_iteration(env, gamma=0.9, theta=1e-6):
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    policy = np.ones((n_states, n_actions)) / n_actions

    iterations = 0
    while True:
        iterations += 1
        V = policy_evaluation(policy, env, gamma, theta)
        new_policy, policy_stable = policy_improvement(policy, env, V, gamma)

        if policy_stable:
            break
        policy = new_policy
        print(f"Итерация {iterations}: политика обновлена")

    print(f"Итерация политики завершена за {iterations} итераций.")
    return policy, V

# Перезапуск с чистыми параметрами
optimal_policy, optimal_V = policy_iteration(env, gamma=0.9, theta=1e-6)

print("\nПроверка политики завершена.")

"""### 6. Визуализация Результатов

```

Давайте визуализируем найденную оптимальную политику и функцию ценности. Для среды `CliffWalking` оптимальная политика обычно отображается в виде стрелок, указывающих оптимальное действие в каждом состоянии, а функция ценности – числовым значением в каждом состоянии.

"""

```

def print_policy(policy, env):
    """
    Визуализирует политику для среды CliffWalking.
    """
    # 0: Up, 1: Right, 2: Down, 3: Left
    actions = ['↑', '→', '↓', '←']

    # CliffWalking-v1 имеет фиксированные размеры 4x12
    nrows = 4
    ncols = 12

    # Форматирование для вывода
    print("\nОптимальная политика:")
    grid_policy = []
    for i in range(nrows):
        row_policy = []
        for j in range(ncols):
            s = i * ncols + j
            if s == (nrows * ncols - 1): # Целевое состояние (последнее
состояние)
                row_policy.append('G')
            elif s >= (nrows - 1) * ncols + 1 and s <= (nrows * ncols - 2):
# Состояния скалы (ряд 3, колонки 1-10)
                row_policy.append('C')
            else:
                optimal_action = np.argmax(policy[s])
                row_policy.append(actions[optimal_action])
        grid_policy.append(" ".join(row_policy))
    print("\n".join(grid_policy))

def print_value_function(V, env):
    """
    Визуализирует функцию ценности для среды CliffWalking.
    """
    # CliffWalking-v1 имеет фиксированные размеры 4x12
    nrows = 4
    ncols = 12

    print("\nОптимальная функция ценности (округлено до 2 знаков после
запятой):")
    grid_values = []
    for i in range(nrows):
        row_values = []
        for j in range(ncols):
            s = i * ncols + j
            if s == (nrows * ncols - 1):
                row_values.append(f"{0.00: 7.2f}") # Ценность цели 0
            elif s >= (nrows - 1) * ncols + 1 and s <= (nrows * ncols - 2):
                row_values.append(f"'CLIFF':<7s") # Отметка скалы
            else:
                row_values.append(f"{V[s]: 7.2f}")
        grid_values.append(" ".join(row_values))
    print("\n".join(grid_values))

# Вывод оптимальной политики и функции ценности
print_policy(optimal_policy, env)
print_value_function(optimal_V, env)

# Закрытие среды
env.close()

```

""### Интерпретация результатов

Визуализация оптимальной политики и функции ценности демонстрирует, как агент научился перемещаться по среде `CliffWalking-v1`.

* **Оптимальная политика:** Агент последовательно избегает области скал (помеченных 'C') и выбирает действия, которые ведут его к цели ('G') с минимальными штрафами. В большинстве безопасных состояний на нижней строке (непосредственно перед скалами) агент выбирает действие 'вверх' (↑), чтобы избежать падения. В других состояниях, чтобы максимально быстро достичь нижней строки перед скалами, агент преимущественно движется 'влево' (←) или 'вверх' (↑) в верхних строках и 'вправо' (→) в первой строке.

* **Оптимальная функция ценности:** Значения функции ценности отражают ожидаемое совокупное вознаграждение, которое агент получит, начиная с каждого состояния и следуя оптимальной политике. Чем ближе состояние к цели и чем безопаснее путь к ней, тем выше (менее отрицательна) ценность состояния. Нулевая ценность в состоянии 'G' подтверждает, что это конечная цель. Очень низкие отрицательные значения в других состояниях показывают накопленный штраф (−1 за каждый шаг) до достижения цели. Состояния 'CLIFF' не имеют числовой ценности, поскольку при попадании в них агент получает большой штраф и возвращается к началу, что уже учтено в вычислениях функции ценности для других состояний.

""